

Fair Drop — Experiment Methodology

1. Principles

1. **Real measurements only.** Every number comes from executing the real server code paths on a deployed drop. The simulator generates *traffic*, the server makes *decisions*.
2. **Label blindness.** The server never sees HUMAN/BOT labels until allocation is final (`population_hash` committed at creation, labels uploaded afterwards and verified against it).
3. **Paired comparison.** NAIVE and FAIR runs of a comparison use the **same** `population_seed`, **same generated population, same op stream generator, same capacity model**. Only `mode` differs.
4. **Reproducibility.** A run is fully determined by: `population_seed`, `ExperimentConfig`, `simulator_version`, `server_version`, `risk_engine_version`, and the allocation `server_seed` (revealed in audit). Re-running logical mode with the same inputs must reproduce allocation results exactly. Real HTTP mode reproduces the population and plan, but not network timing (declared).
5. **Declared assumptions.** Capacity model, challenge oracle, and confirm probabilities are part of the stored config and are shown beside every result.

2. Client profiles

Parameters are drawn from the profile using a per-participant PRNG `Random(H(population_seed, participant_index))`.

Profile	Accounts/operator	Sessions/account	First arrival (after open)	Request amplification	Retry policy	Telemetry model	Challenge oracle pass	Confirm prob.
HUMAN	1	1 (5 % have 2)	lognormal, median 20 s (wall-scaled to window)	1	up to 3, backoff 2–8 s, respects retry_after	human distributions (varied modality incl. keyboard/touch-only)	0.97	0.90
SIMPLE_BOT	1	1	uniform 0–200 ms	1–3	fixed 1 s	none / empty summaries	0.05	1.0
FLOOD_BOT	1	1	0–50 ms	request amplification (default 200)	ignores retry_after	none	0.05	1.0
RETRY_BOT	1	1	0–100 ms	1	retry_amplification (default 50) at 100 ms	none	0.05	1.0
SLOW_BOT	1	1	uniform over window	1	≤ L1 limits	minimal plausible	0.30	1.0
MULTI_SESSION_BOT	K (default 20)	3	0–100 ms	5	10	shared device_ref across a subset of accounts, shared network_ref	0.10	1.0
HUMAN_LIKE_BOT	1	1	lognormal like humans	1–2	human-like	sampled from human distribution with small artifacts	0.60 (solver farm)	1.0
MIXED_ATTACK	—	—	composite: mix of above per profile_mix	—	—	—	—	—

3. Scenarios

Scenario	Population (default)	Purpose
normal	5,000, 100 % HUMAN	Baseline sanity, HAR=N/E, J inside null band
flash_crowd	50,000, 98 % HUMAN, 2 % SIMPLE_BOT, arrivals compressed into first 10 s	Load + speed independence (AAR, J)
request_flood	50,000, 95 % HUMAN, 5 % FLOOD_BOT	Volume independence (RVA, VS)
retry_storm	50,000, 95 % HUMAN, 5 % RETRY_BOT	Retry collapse, idempotency
multi_session	50,000, 95 % HUMAN, 5 % of participants as MULTI_SESSION_BOT accounts	Dedupe + honest Sybil exposure (OWR)
low_and_slow	50,000, 97 % HUMAN, 3 % SLOW_BOT	Shows no advantage is gained (not "detected")
human_like	50,000, 95 % HUMAN, 5 % HUMAN_LIKE_BOT	Worst case for detection. Lottery bound
mixed_attack	50,000, 80 % HUMAN, 20 % mixed bots	Headline demo
race_reserve (REAL_HTTP)	500 winners + 2,000 attackers, 200 concurrent conns firing reserve/confirm simultaneously	Integrity under true concurrency
amplification_sweep	request_flood at amp $\in \{1, 10, 100, 1000\}$	VS(A) headline

Configurable knobs (all stored): total_participants ($\leq 50,000$), profile_mix (human % / bot %), request_amplification, retry_amplification, concurrency, traffic_rate_rps, attack type, population_seed, timings, risk.enabled, controls.enabled, capacity model.

4. Simulator modes

4.1 Logical simulation (scale: up to 50,000)

1. Generate the population deterministically. Compute `population_hash = SHA-256(canonical JSON of [index, account_ref, profile, operator_ref])`.
2. `POST /sim/experiments (mode, config, seeds, hash) → experiment_id, sim_token`.
3. Build a **global event timeline** over a virtual clock: every attempt of every participant (incl. amplification as `c` counts, retries, status polls at `next_poll_ms`, telemetry windows, challenge outcomes from the oracle, reserve/confirm by `confirm_probability`).
4. Apply the **capacity model** identically in both modes: per `per_tick_ms` tick, the server accepts at most `capacity_per_tick` attempts (FIFO by virtual time), and the rest get `503 OVERLOADED` and are re-scheduled per the profile's retry policy. *This is applied server-side in the DO* so it is part of the measured system, and the parameter is declared.
5. Stream ops in `batch_seq` order ($\leq 1,000$ ops/batch). Each batch carries `virtual_now`. The DO advances its clock monotonically and runs time-driven transitions (close registration, expiries) at batch start.
6. Reactive behavior: simulator agents react to `OpResult` s (e.g. winners proceed to reserve), so the timeline is generated incrementally, tick by tick, not precomputed blindly.
7. `finalize → labels upload → server computes results`.

Request accounting: each amplified attempt passes through L1–L4 + risk individually inside the DO. L0 (edge) isn't exercised in logical mode. This is declared, and L0 is tested in real HTTP mode.

4.2 Real HTTP load (scale: bounded by quota, default $\leq 20,000$ requests/run)

- `asyncio + httpx.AsyncClient(http2=True)`, bounded semaphore = `concurrency`, token-bucket pacing = `traffic_rate_rps`.
- Each simulated client gets its own cookie jar via `POST /session` with `X-FD-Sim-Token` + `X-FD-Sim-Client` (virtual network). The drop uses the **wall clock**, short timings (e.g. registration 60 s, hold 20 s).
- Uses the public consumer endpoints, exercising Worker auth, L0 edge limiting (keyed by virtual network), DO, and the real network path.
- Client-side latency is recorded in log-bucket histograms and pushed every 2 s to `/client-metrics`.

- Purpose: validate that logical-mode conclusions hold on the real path, and measure latency and race integrity.

5. Procedure for a comparison

```
for scenario in selected:
  pop_seed = fixed per scenario (published)
  for repetition r in 1..R (default R=5; allocation server_seed differs per run):
    run NAIVE (logical) → results_naive[r]
    run FAIR (logical) → results_fair[r]
  run 1 REAL_HTTP spot-check at reduced scale for FAIR and NAIVE
report per metric: per-run values, mean, 95% CI across repetitions, pooled Wilson CI
```

Ablations (FAIR only, same population): `risk.enabled=false`, `controls.enabled=false`, both disabled. This shows the lottery alone yields $RVA_{eligible} \approx 1$ (1-9), and shows what detection adds on top.

6. Live dashboard data

The DO maintains counters and broadcasts a snapshot every 500 ms: total/human/bot participants (human/bot shown only **after** labels upload; during the run the dashboard shows the simulator-reported *planned* mix, clearly labelled "planned"), total requests, req/s, blocked, throttled, challenged, deduplicated, eligible, lottery entries, allocated, waitlisted, active holds, confirmed, expired holds, oversells, duplicate allocations, inventory drift, P50/P95/P99, error rate.

Event feed (sampled $\leq 50/s$, most-severe-first): `ACC-3f2a → rate limited (L1 register)`, `ACC-3f2a → duplicate request collapsed`, `ACC-91c0 → challenge issued [NET_BURST, NAV_SKIP]`, `PAR-0021 → eligible`, `Allocation batch → started (eligible=47,912, seed commitment 9ac1...)`, `Allocation batch → completed (500 winners, result_hash 4be0...)`, `Hold expired → ticket #212 AVAILABLE → promoted waitlist #1`. Subjects are opaque refs, and labels are added only on replay after the labels upload.

7. Threats to validity (stated in the results page)

- **Synthetic behavior:** detection performance reflects our generators, not real bots.
- **Capacity model:** NAIVE unfairness magnitude depends on the declared capacity; we show results for ≥ 2 capacity settings.
- **Logical vs real:** logical mode bypasses L0 and network timing. Real HTTP spot-checks cover these.
- **Sample noise:** with 500 winners, group rate CIs are $\pm \sim 1-2$ pp. We report CIs and the Jain null band.
- **Single region/DO:** latency numbers are for one deployment from one client location.